1 Introduction to distributed systems

Contents

1.1 Motivation
- 1.1.1 Types of systems
- 1.1.2 A definition of distributed system
- 1.1.3 Perspectives

1.2 Properties of distributed systems
- 1.2.1 Transparency
- 1.2.2 Scalability
- 1.2.3 Reliability anf fault tolerance
- 1.2.4 Consistency
- 1.2.5 Other properties

1.3 Distributed applications
- 1.3.1 Objectives of distributed applications
- 1.3.2 Environments for distributed applications

1.4 Hardware support
- 1.4.1 Computational elements
- 1.4.2 Network infrastructure

1.5 Operating system support

1.6 Communication support
- 1.6.1 Communication models
- 1.6.2 Comunication based on message passing

1.7 Open systems

1.8 Summary

1.9 Exercises

Appendix A: Remothe procedure calls

Appendix B: Remothe method invocation

1.1 Motivation

Distributed systems represent another step in the evolution of computer systems, understood from the point of view of the needs that applications pose and the possibilities that technology offers. Before providing a definition of a distributed system, it will be interesting to present, through historical evolution, the concepts that have led to current distributed systems, characterized by the physical distribution of resources in interconnected machines.

We will use here the term **resource** with general character to refer to any device or service, hardware or software, capable of being shared.

1.1.1 Types of systems

From a historical perspective we can talk about different models that determine the functionality and structure of a computer system, the characteristics of the operating system as a resource manager, and its field of application and use:

- **Batch systems**. These are the first operating systems, which allowed deferred and sequential processing of data supplied in punch card packages. Nowadays, on multiprogrammed systems with interactive user interfaces, the batch processing makes sense in intensive calculation applications, for example in supercomputing.

- Centralized **timeshare systems**. It was the next step, in the mid-60s. The objective is to increase the efficiency in the use of the CPU, a resource that was expensive and scarce at that time, decreasing the response times to the users, who operate interactively. The resources are centralized and the system is accessed from terminals.

- **Teleprocessing systems**. They differ from the previous model in that the terminals —most recently, personal systems— are remote and access a central system using a network infrastructure (for example, the telephone network) and a communication protocol normally of proprietary type. The central system monopolizes the management of resources. Examples of applications that this model solved are the reservation systems and banking transactions.

- **Personal systems**. The motivation of this type of systems was to provide a dedicated system for a single user, which was possible thanks to the cheaper hardware by the emergence of the microprocessor at the

beginning of the 80s. Precisely, the reduced cost is its fundamental characteristic. The operating system of a personal computer (PC) is, initially, single-user: it lacks protection mechanisms. Although, for simplicity, the first operating systems were monoprogrammed (MS-DOS), the hardware upgrade soon allowed to support multitasking systems (Macintosh, OS/2, Windows 95/98), and even operating systems designed for timeshare, such as UNIX and Windows NT[1]. A personal system has its own local resources. Initially, these were the only accessible resources, but nowadays the situation has changed. On the other hand, hardware evolution has taken personal computers to mobile versions (portable PCs and other devices such as PDAs and mobile phones).

- **Networked systems**. In the evolution of teleprocessing, terminals were gaining computing capacity and functionality to become autonomous systems. The concept of a central computer disappears. Now we have to talk about a set of computers that connect to each other using a network infrastructure. A machine that provides access to a certain resource is the **server** of that resource. **Clients**, who can have **local resources**, access a **remote resource** by request to the corresponding server. There are proprietary network protocols, which restrict interoperability to machines of the same type that make up a **local network**, such as Novell for PCs, or AppelTalk for Macintosh. However, the development of common protocols, such as TCP/IP, has allowed the interconnection of machines regardless of their characteristics and operating system (interoperability), extending the scope of these systems to **wide area networks** and enabling the emergence of the **Internet**.

- **Distributed systems**. The resources of different networked machines are *integrated* in such a way that the local/remote duality disappears. The fundamental difference with networked systems is that the location of the resource is *transparent* to applications and users, so, from this point of view, there is no difference with a timeshare system. The user accesses the resources of the distributed system through a graphical user interface from a terminal, disregarding its location. The applications execute a

[1] This type of systems was known around 1990 as *workstations*. Nowadays any personal computer is capable of supporting them and this distinction is no longer made. The PC has become a standard component that, as we shall see, is used interchangeably as a personal system, as a server of shared resources, or even as an element of a high-performance system (*cluster computing*).

system call interface as if it were a centralized system, for example POSIX. A remote invocation service (for example, to procedures, RPC, or to objects, RMI) resolves the access to non-local resources using the network interface. Distributed systems provide transparent sharing of resources, facilitating access and management, and increasing efficiency and availability.

The distributed system model is the most general, so although the same integration for all types of resources has not been achieved commercially, the trend is clear in favor of this type of system. The other motivation is the relationship of costs to which technological evolution has led in recent years. Today there is a standard hardware of low cost, the personal computers, which are the basic components of the system. On the other hand, the communication network, unless great features are required, is not a big economic problem either, being able to use already existing wired infrastructure (Ethernet, the telephone network, or even the electrical network) or wireless.

1.1.2 A definition of distributed system

We can now venture a definition of distributed system:

(1) a set of computers

(2) interconnected

(3) which share a state,

(4) offering a single system vision.

Leaving aside the discussion on the meaning of the term *computer*, the characteristic (2), which differentiates them from personal systems, is shared by distributed systems and networked systems. Characteristic (3), on the other hand, is exclusive to distributed systems, being its consequence (4) the single system vision[2], which shows resources in a homogeneous way, hiding their distribution: *the user and the applications do not see a network, but a system indistinguishable from a centralized one*. While a networked system can be defined as a set of systems with independent states, in a distributed system a **global state** is defined.

[2] *Single System Image*, SSI [BAK00].

The topology and the physical attributes of the network are hidden by the network protocols, while the architecture of each machine is hidden by the operating system. As the components of a distributed system can be heterogeneous, a software layer (often called *middleware*) is required to provide the single system vision. We will return to this point when discussing the structure of the distributed system.

1.1.3 Perspectives

The technological development in a set of areas –processors, information storage, networks (in particular wireless), energy storage, screens– is introducing new devices that open perspectives for new applications in the world of distributed systems. An evolution, initiated in the nineties of the last century from the introduction of portable equipment, takes us to what we call **mobile computing**. The mobile device leaves the local network to discover new resources. It has to adapt to changing network conditions and sometimes operate in disconnected mode. *Ad-hoc* networks that require specific protocols such as Mobile IP [PER98] can be spontaneously constituted. The advantages of wireless communications become clear.

One step further is the **ubiquitous systems** (*pervasive systems*) [COU05 §16]. The dizzying development of hardware technology is taking the miniaturization of computing devices to new products. Portable computers (which can be transported) become ubiquitous when their presence is no longer perceived (*technology that disappears* [WEI91]). From another point of view, any device acquires the computing and communication capacity of a computer. Current examples or of a near future of this type of devices are the mobile telephones, the navigation systems of automobiles, the smart cards, the sensors (*motes*) and many other devices that will arise. A ubiquitous environment is by nature changing. Resources and services are discovered and configured dynamically, and applications and user interfaces adapt to the environment.

In addition to the problems posed by distributed systems and mobile computing, ubiquitous systems offer new challenges that involve numerous technological fields: hardware; networks and communication protocols (in particular for device discovery, such as Jini[3] and UPnP[4]), user interfaces (with new devices and forms of interaction), and security. Ubiquitous systems have a wide range of application in very diverse environments: home automation

[3] http://:www.jini.org

[4] http://www.upnp.org

(domotics), commerce and industry, welfare services, education, leisure and tourism, automotive, robotics…

The ubiquity of the devices has led to new paradigms also in the field of Internet. Thus, nowadays, *cloud computing* services have been extended, offering Internet resources (in the *cloud*) to transparently support environments and applications understood until recently as linked to physical entities (for example, a PC or a disk), as it is the case of repositories of documents, agenda or email. The objective is the *virtualization* of the information spaces and the mobility of the users regardless of the device. Dropbox, Google Drive or Gmail are very popular examples of this phenomenon.

On the other hand, the fact that any device or resource can connect to the Internet makes it a candidate to be offered as a web service, which constitutes what is being called the *Internet of Things*.

1.2 Properties of distributed systems

A distributed system that intends to offer a *single system vision* must comply with the properties presented below[5].

1.2.1 Transparency

The essential objective of a distributed system is to provide the user and the applications with a view of the system resources as managed by a single virtual machine. The physical distribution of resources is transparent. Different aspects of transparency can be described:

- Of **identification**. The **namespaces** of the resources are independent of the topology of the network and the distribution of the resources themselves. In this way, an application can refer to a resource with a name regardless of which node it runs on (see the right column of Figure 1.1). For example, in NFS a remote file system is mounted at a point in the local file system. For an NFS installation to provide applications identification transparency it depends on all client nodes having the remote system mounted on the same mount point, which is usually the responsibility of the administrator.

[5] They should be understood as desirable properties. As we will see repeatedly, the commitment to performance causes in practice that some of the properties are relaxed.

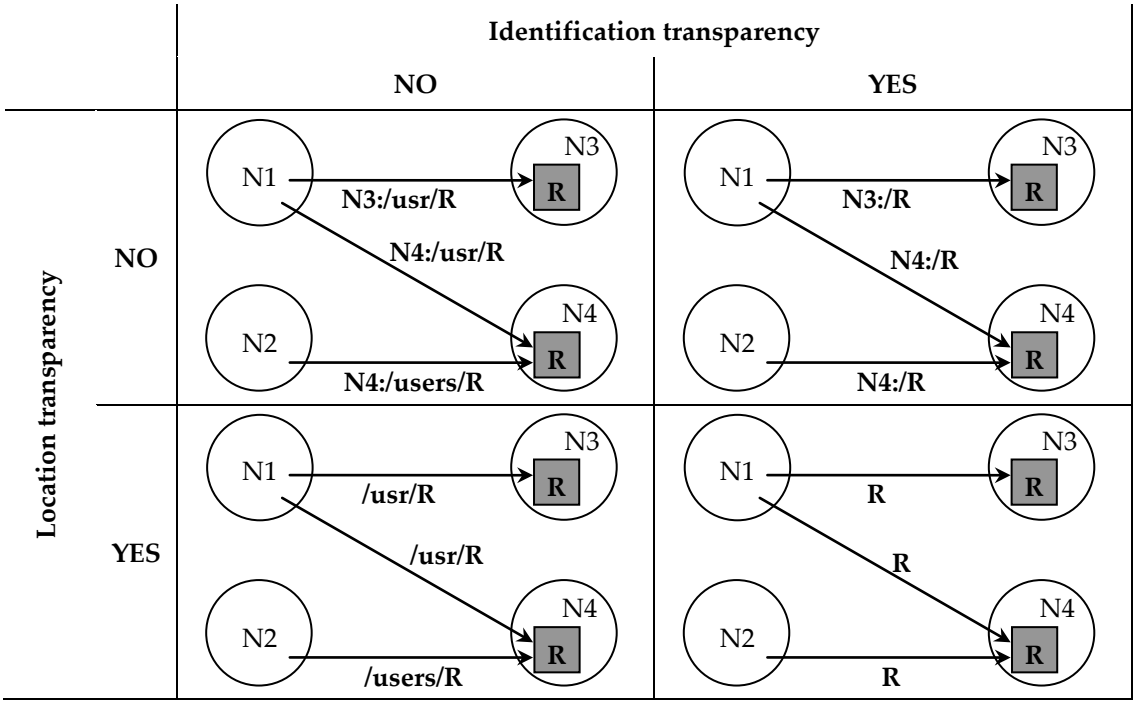


Figure 1.1. Identification transparency and location transparency

- Of the physical **location** of the resources. Neither the users nor the applications know in which node the accessed resource resides, or if this is local or remote. This also implies that resources can **migrate** between nodes without the applications being affected. The difference between location transparency and identification is summarized in Figure 1.1. In NFS, we can migrate a remote file system from one node to another and the location transparency will be preserved if the mounting tables of the client nodes are updated properly.

- Of **replication**. Neither the users nor the applications know how many copies there are of each resource, or whether copies of the resource are added or deleted. In NFS clients manage local caches of the remote files. *Caching* is a restricted form of replication that requires some form of validation (in NFS through polling) by clients to preserve consistency[6]. Even so, the UNIX semantics that NFS intends to offer to applications can sometimes be compromised when several clients write on the same file. Other more general forms of replication, as will be seen, are more complex to manage.

[6] We will talk later about *consistency* as a specific property.

- Of **parallelism**. An application can be executed in parallel, without the application having to specify it, and without consequences on the execution, except for performance issues. This property affects the systems that allow processes and memory to be distributed. In the case of a file system, it is only relevant when applications temporarily block access to files, which is specified explicitly (by means of *lock* and *unlock* primitives). The latest versions of NFS include this mechanism.

- Of **sharing**. The fact that a shared resource tries to be accessed simultaneously from several applications has no effect on the execution of the application. As we have said, in NFS this property can be affected by the existence of caching, particularly if the periods of the validation polling are high.

1.2.2 Scalability

One of the characteristics of distributed systems is their **modularity**, which allows great flexibility and enables their **scalability**, defined as the ability of the system to grow without increasing its complexity or diminishing its performance. One of the objectives of the design of a distributed system is to extend the scalability to the integration of services.

Scalability presents two aspects. The distributed system must (1) provide sufficiently large **namespaces**, so that they do not imply an inherent limitation, and (2) maintain a good level of performance in accessing resources when the system grows.

- **Namespaces**. Namespaces, as in centralized systems, can identify objects of different nature, such as files, processes, variables, or even memory addresses (in *distributed shared memory* systems, DSM). In the case of linear spaces, such as memory, there is an inherent limitation associated with the size of the name, so that today it is reasonable to raise the insufficiency of 32-bit address spaces. In other cases, the namespaces are hierarchical and therefore scalable by nature.

- **Complexity/performance**.[7] The growth of a distributed system can introduce bottlenecks and latencies that degrade its performance. In addition to the increase in communication costs due to the increase of the physical distance between the components of the system, the structural

[7] It refers to both the structure of the system and the algorithms.

complexity of the distributed algorithms is often more than linear with respect to the size of the system, as we will see throughout the course. It is necessary, therefore, to establish compromises between system size, performance and complexity.

1.2.3 Reliability and fault tolerance

The reliability of a system can be defined as its ability to perform correctly and at all times the functions for which it has been designed. The reliability is specified in two aspects:

- **Availability**. It is the fraction of time that the system is operative. The main parameter to measure the availability is the mean time between failures (MTBF), but we must also consider the repair time. Availability can be increased in two ways: (a) using higher quality components, and/or (b) with a design based on the replication of components that allows the system to continue operating even when one or more of them fail. Both alternatives increase the cost of the system; however, in the current technological state, replication is, in general, less expensive. Distributed systems inherently provide the replication of some resources (for example, processing units), while others that are normally shared (for example, a file server) can be replicated to increase availability. On the other hand, the absence of failures in the components of a system, both hardware and software, can never be guaranteed, so that, beyond limits, replication is necessary to continue increasing availability, since the probability of failure decreases as an exponential function of replication. For example, given a failure probability of 1% in a component (in a given period of time), if we assemble a replicated system with four identical components, the probability of failure in that period of the four components would decrease to 0.000001%[8].

- **Fault tolerance**. Even with high availability, a failure at a certain time can have disastrous consequences. Think of critical real-time systems that control vital devices (for example in medicine, nuclear power plants…). That is, although replication increases availability, it does not guarantee the continuity of the service in a transparent way. Fault tolerance expresses the ability of the system to continue operating correctly when

[8] This must be qualified: on the one hand, there can be correlation in the failure of the individual components due to several causes, such as design errors or natural catastrophes. On the other hand, the interconnection of the components is a source of additional failures.

any of its components fail, masking the fault to the user or the application. Therefore, fault tolerance involves (1) detecting the failure, and (2) continuing the service, all in a transparent manner for the application (**fault transparency**).

We can say about our example that, in the absence of service replication, NFS does not provide fault tolerance. On the other hand, other file systems have been proposed that do have fault tolerance as a design objective, as is the case of AFS [SAT90].

1.2.4 Consistency

The distribution of resources introduces important benefits. On the one hand, it contributes to increasing performance through parallelism and promoting access to local copies of the resource (decreasing communication costs). On the other hand, as we have just seen, replication increases availability, being the basis for providing fault tolerance. However, distributing resources brings some problems. On the one hand, the interconnection network is a new source of failures. In addition, the security of the system is more vulnerable to unauthorized access. But the problem of greater complexity is the **management of the global state** to avoid situations of inconsistency between the components of the system. This is a fundamental aspect in the design of the distributed system, so we will discuss it below.

The problem lies in the need to maintain a global state consisting of a system with several components, each of which has its own local state. The nodes of the system are physically distributed, so that the management of the global state depends strongly on the communication mechanisms, in turn supported by a network subject to failures. As we will see, consistency management can be based on good synchronization between node clocks or on event ordering mechanisms (logical clocks). The physical distribution makes, in general, unfeasible the use of a global clock that provides absolute time references, which would allow a total ordering of the events and, therefore, of the state transitions in each node.

Thus, the maintenance of a strict consistency requires a strong support that implies a great load of additional communication between the nodes of the system, so it is often preferable to relax the consistency to maintain the performance at an acceptable level, according to the needs of the applications.

1.2.5 Other properties

In addition to the (debatable) selection of characteristic properties of the distributed systems described in the previous sections, it is evident that distributed systems must also provide the common properties of networked or centralized systems, such as performance, security and privacy, or ease of use.

It is noteworthy that performance is in general a property that is antagonistic to the rest, which generally leads to the implementation of compromise solutions. For example, in NFS the minimization of the cache validation period would make it almost completely transparent the existence of caches, at the expense of increasing network traffic and consequently latencies.

1.3 Distributed applications

First, we must distinguish between distributed applications and parallel applications. A parallel application is one that can be divided into tasks that are executed concurrently in different processing elements, in order to reduce the time of completion. Most applications can run in parallel, depending on certain computation schemes (for example, following a *pipeline* model) that depend on the nature of the application and the hardware on which it will be executed. The tasks of a parallel application are distributed among the processing elements following criteria such as the load of each element and the communication costs. It can be said that the simultaneous execution of tasks is the essential objective of a parallel application. Distributed applications have more diverse motivations and are applied in more varied environments.

1.3.1 Objectives of distributed applications

If in the parallel applications the performance is the fundamental objective, a distributed application can present very diverse motivations:

- **High performance**. A parallel application can also be distributed. For example, a local network can be used to distribute the tasks between the nodes of the network in order to take advantage of the available computing resources (usually low cost PCs) to reduce the completion time. Precisely, this type of computing scheme (*cluster computing*) today offers an excellent performance/cost ratio and is expanding compared to traditional supercomputers. Although systems based on message passing (for example, MPI) with non-transparent distribution mechanisms are still used today, the trend is clear towards the

transparent integration of computing resources, and in this sense new products are appearing (for example, MOSIX[9]). Distributed shared memory (DSM) is one of the challenges to provide full integration. In wide area networks we talk about *grid computing*. In this case, the availability of resources for the application is open and includes scattered computing units that are currently idle. An example of software to support grid computing is the *Globus Toolkit*[10].

- **Fault tolerance**. In other applications, distribution is dictated by criteria such as the integrity of information. Thus, in a banking system it is necessary to keep replicated information about the status of customer accounts on different servers, because the risk of losing information due to the failure of a machine is unacceptable due to the consequences that would entail. In these systems it is critical to get a consistent update of the replicas. Nowadays, mainly for performance reasons, commercial systems use very conservative and not very transparent techniques, but, as we will see, the theoretical foundations for a transparent management of replication are well established.

- **High availability**. There are applications where the distribution is made to bring the information closer to the user and reduce response times. In the simplest cases, replication techniques are used that take into account the geographical distribution (*caching* and *mirroring*). Consistency in the update is not usually a critical aspect; instead scalability matters a lot. Nowadays, *peer-to-peer* systems are widespread, characterized by their great scalability by avoiding server bottlenecks, offering resources availability in a practically indiscriminate manner. An example are content distribution networks, such as BitTorrent.

- **Mobility**. The abundance of physical devices (personal computers and laptops, tablets, mobile phones, etc.) introduces an additional difficulty for accessing user information, so that the user does not have to manage the updating of information on each device. For example, an email deleted from the mobile phone should appear as deleted when the user subsequently accesses his email from a personal computer. It is essential to separate the information from its support, conveniently managing the updates. More and more work is done on virtual information spaces instead of on specific physical devices, which become mere *caches* of the

[9] http://www.mosix.cs.huji.ac.il/

[10] http://www.globus.org

user's information space. Thus, the user moves from one device to another and accesses the space of its information in an updated and consistent manner. Examples of current products are Google *Gmail* for email and *Dropbox* for documents.

- **Ubiquity.** Sometimes resources are inherently distributed. The user moves in an environment with (ubiquitous) resources not foreseen a priori, and the application tries to offer an *intelligent* behavior according to the needs of the user and the nature and availability of the resources. This is the case of the applications of *Ambient Intelligence* (*AmI*), a field that is creating enormous expectations.

1.3.2 Environments for distributed applications

The availability of infrastructures such as those cited above offers several scenarios where distributed applications can be deployed. It can be said that, at present, the Internet provides the most general environment, but there are applications that make sense in more specific environments.

1.3.2.1 Internet

Undoubtedly, the *World-Wide-Web* (for short, *WWW* or *Web*) is the star application of the Internet, to the point that nowadays it is also the form of common access to other Internet applications, such as email and file transfer. It is based on the existence of a strong communication structure, which includes a high-backbone infrastructure where the subnets and ISPs (Internet Service Providers) are connected, which provide access to this infrastructure to the private users using various media (for example, the ubiquitous traditional telephone wiring). The Web has imposed HTTP as a common access protocol.

The distributed applications that can be deployed on the Internet are very varied, although limited in practice by aspects of performance and security. The so-called *peer-to-peer* applications are a clear example of high availability applications that are deployed on the Internet. Another example is the systems for *grid* computing, which on the basis of a peer-to-peer structure offer a transparent management of the set of distributed resources available in the network. Finally, today, *cloud computing* services are of great importance, as we have already mentioned, both for the business world and for individuals.

1.3.2.2 Intranets

An intranet is basically a restricted Internet environment. The same protocols and access means are used as on the Internet, although the access is limited to a specific administrative domain or a set of them (for example, in a company). An intranet can be composed of several subnets, and these can be integrated into the Internet, although access from the outside or the output from the intranet can be restricted by specific servers (*firewalls*), which act as filters applying criteria at different levels of the protocol stack.

In principle, an intranet supports the same type of applications as those that can be deployed on the Internet. In addition, high-performance applications, such as cluster computing, or fault-tolerance applications, usually make more sense on an intranet (or on an intranet LAN), since, in general, the communication costs can be bounded.

1.3.2.3 Ubiquitous environments

In a ubiquitous system, applications are not subject to specific administrative or network areas, such as intranets, but they also do not operate in an unlimited way in the Internet. Thus, a user's device (for example, a mobile phone in the near future) operates in the specific *physical* environment in which it is located (for example, the user's home or an airport) according to the services it discovers in that environment, and adapts to the available infrastructure in the most efficient way possible. For example, the same telephone call from a user leaving its home to the airport could use different infrastructures: (1) the ADSL line at home to which the telephone is connected by means of an access point accessed by Bluetooth, (2) mobile telephony offered by an operator during the trip to the airport, or (3) an Internet access service offered by another operator via WiFi, cheaper than mobile telephony, once it arrives at the airport. Communication modes would dynamically switch, optimizing the quality and cost of communication. Note that to achieve this type of operation a prerequisite is that the user's device supports several modes of wireless communication between it and the infrastructure offered by the physical environment. However, the biggest challenge is to support changes in a transparent way[11].

Of course, a ubiquitous environment can also provide access to Internet or intranet services, so this type of environment can be considered the most

[11] The systems that provide this type of transparency are called *seamless*.

general, and, as mentioned, pose great challenges in terms of infrastructure, hardware devices (miniaturization, autonomy…) and standardization, and offer high expectations (*AmI* applications).

1.4 Hardware support

We will distinguish here between the devices that support the computation in a distributed system and the network infrastructure that allows their interconnection.

1.4.1 Computational elements

The definition of distributed system that we have provided before (set of interconnected computers that share a state) is ambiguous enough to raise discussions, for example (1) what is meant by a computer, and (2) how states are defined. To answer these questions we need to know the alternatives in terms of computer architectures. In accordance with Flynn's classic criteria, architectures can be classified according to data and/or instruction parallelism, obtaining the four groups of Table 1.1.

On this classification, it could be qualified that the current processors (segmented and superscalar) also perform some kind of parallel process. But, according to our purpose of specifying which architectures can support *distribution* (of any type of resource), we must focus only on the MIMD category, machines with several processing units[12]. By generality, we will introduce the term **node** to refer to each of the processing units.

[12] We will understand as the processing unit the element that is capable of executing machine instructions, that is, the *processor* in classical terminology.

	a data simultaneously	multiple data simultaneously
an instruction simultaneously	SISD Von Neumann classic architecture	SIMD vector processors
multiple instructions simultaneously	MISD not implemented	MIMD multiprocessors, multicomputers, networks

Table 1.1. Flynn's classification

The MIMD category is usually subdivided into subgroups according to two criteria: the degree of coupling (if each node has its own physical memory address space or not), and how the nodes are connected (shared bus or interconnection network). Table 1.2 shows what types of systems this classification leads to.

	degree of coupling	
	shared physical memory space	independent physical memory spaces
shared bus	typical multiprocessor servers	multicomputers, LANs
interconnection network	UMA and NUMA multiprocessors	multicomputers, WANs

Table 1.2. Types of MIMD architectures

Again, on this classification it is possible to ask whether all the subgroups generally admit resource distribution. To refine a bit more, we will identify three groups of elements that can be distributed among the nodes:

- Processing
- Memory space
- I/O space (files and devices)

In multiprocessor systems, memory and I/O are by definition centralized resources. In these machines it would only make sense to talk about distributed processing. The nodes share the state of I/O and memory, the physical distance between processors is small and the clock is unique. In the current multiprocessors, each processor has its own cache memory, but consistency is

guaranteed by hardware protocols. A **multiprocessor operating system** apparently does not introduce great novelties[13]. Memory, I/O and files are managed as in traditional operating systems. Only the management of the processors and the allocation of processes pose new needs. Therefore, today multiprocessors are not considered distributed systems.

On the other hand, in multicomputers and networks, all resources, including the clock, are (or can be) distributed[14]. It is necessary to implement mechanisms and services that ensure consistent global states and provide the vision of the single system to an acceptable degree. In practice, most of the distributed systems of today are based on consumption computing elements, mainly PCs, and the tendency is, as we have seen, that the elements of computing are integrated into all types of devices, such as telephones or cameras (or, from another point of view, that every type of device acquires capacity to serve as a node in a distributed system), and that these elements are integrated into the system through some type of standard network infrastructure

1.4.2 Network infrastructure

As discussed above, today distributed systems are based on the existence of a communications network that connects standard computing devices. Currently, the industrialized world has extensive trunk infrastructures (*backbones*), usually of fiber optic, which has allowed the deployment of the Internet. Satellite links are also used to access remote places. The problem of access to user points (metropolitan area networks, MAN) has been solved through the efficient use of the old telephone wiring. For specific local networks Ethernet has been used, which allows very high speeds (up to 1 Gbps).

In recent years, wireless technologies have experienced a great advance, to the point that they can compete (as in the case of IEEE 802.11 WiFi) with Ethernet in local networks. There are also solutions for higher geographical areas. Finally, in ubiquitous systems, short-range wireless networks (personal networks, PAN) become important. Table 1.3 illustrates some characteristics of current network technologies.

[13] This is only in appearance. An efficient support for the multiprocessing has important implications in the structure of the operating system.

[14] In multicomputers, the I/O space can be either single or distributed, but generally we will assume the last.

	Type	Example	Typical range	Bandwidth
Wired networks	PAN	USB	1 m	12 – 480 Mbps
	LAN	Ethernet	1 Km	10 – 1000 Mbps
	MAN	ATM	10 Km	1 – 150 Mbps
	WAN	Internet	All the world	0,5 – 600 Mbps
Wireless networks	PAN	Bluetooth	10 m	0,5 – 2 Mbps
	LAN	WiFi	100 m	2 – 54 Mbps
	MAN	WiMAX	10 Km	1,5 – 20 Mbps
	WAN	HDSPA	All the world	up to 14 Mbps

Table 1.3. Examples of network technologies

1.5 Operating system support

We have limited the concept of distributed system as applicable to those systems composed of nodes with their own memory and I/O spaces. Each node has its own operating system and basic network services. This description is consistent with that of computer networks, but may also include multicomputers.

In general, the nodes will be heterogeneous in terms of hardware and operating system. A first step is to provide a set of specifications that allow defining common interfaces to build on them the services of the distributed system and provide the single system vision.

Regarding the distribution of resources, it is possible to request from the operating system (1) compatibility, to allow the development of the distributed services, and (2) to provide the adequate flexibility to locate and manage the services efficiently. However, classical systems, such as UNIX, monolithic, are not designed according to these objectives. They offer a unique system call interface, integrate all kinds of services in the kernel and provide predetermined management policies.

The classic approach to build a distributed system in a network is to install on the operating system of each machine the necessary mechanisms to provide access to remote resources according to the client-server scheme. This approach provides little flexibility, since the server of a resource usually requires running a particular operating system. On the other hand, on the client side, the execution of applications that must be supported by specific platforms is usually required. Such a rigid scheme leads to associating specific machines

with specific services and applications, to the detriment of flexibility and location transparency.

A solution to allow any node in a network to support any service or application indiscriminately is the **emulation** of a guest system on a host operating system. This provides **multiple personality** to each node[15], allowing the coexistence of applications and services, either in native mode, or emulated. There are products for Windows and Linux, such as *VMware*, *VirtualPC* and *win4lin*, which provide the emulation of a hardware environment on which the guest operating system is installed. The success of these products is that they use a technique, *virtualization*, which allows executing most of the code in native mode, emulating only the calls to the guest operating system, so the strong loss of performance inherent to the emulation can be assumed by the current machines.

However, in the early years of distributed systems, hardware emulation was practically unaffordable, and the very fact of having to store and load several complete operating systems, too expensive. All this led some 25 years ago to a revision of the structure of the operating system. The premise was to remove from the space of the operating system the services that could be distributed, together with their management policies, leaving inside, for its execution in protected mode, only the basic functions of hardware control (basic management of memory and interruptions) and the support for context changes and communication. This core operating system is called **microkernel**, and provides the appropriate basis for the distribution of services, allowing the selective implementation on it, in the user space and according to the client-server model, of the specific services offered by the system, as independent modules. The system call interface is implemented as a subsystem in user space, which allows, naturally and with little overhead, to offer multiple personality and concurrently support applications and services of different operating systems. The most widespread example of microkernel is Mach 3.0, the basis of some UNIX-like commercial systems, such as OSF/1, or Apple's current OS X for Macintosh.

[15] In practice this implies that you can open several windows with different operating systems in the same terminal.

More and more frequently the current mobile devices (in particular the so-called *smart phones*) support small versions of commercial operating systems (Mac, Windows), or specific systems (such as Android, from Google) with no disdainful features. Currently these systems have the capacity to support a Java virtual machine, a Web browser, multimedia applications, and control a variety of devices, such as a touch screen, a camera, a GPS receiver, and all kinds of wireless communication interfaces which makes them extraordinarily versatile. This sector is undoubtedly the hottest scenario for the development of operating system technology.

Finally, at the end of the miniaturization, autonomous devices equipped with sensors and wireless communication capacity designed to be configured in *sensor networks* are being proposed. Each of these devices (*motes*), disseminated more or less randomly in diverse environments (for example, an ecosystem), participate both in the collection of information and in the routing of the messages from the nearby motes. There are specific operating systems to support these functions with minimum energy consumption, such as TinyOS[16].

1.6 Communication support

We must differentiate between physical distribution of memory[17] and **communication model**. The degree of coupling will determine the support needed to implement the communication model, but it can be based on both shared memory and message passing. The chosen model determines part of the characteristics of the distributed system, but the physical distribution of the memory is hidden, except for performance purposes.

1.6.1 Communication models

On shared memory, the mechanisms of shared variables whose access is synchronized by mutual exclusion locks or other primitives (condition variables, readers-writers locks), have been studied for a long time and are well established. The same can be said about message passing, which implements

[16] http://www.tinyos.net

[17] The generalization of the local cache memories to each processor means that, nowadays, all the MIMD machines present some form of physical distribution of the memory, although the multiprocessors, where the cache coherence is guaranteed by the hardware, are considered as of shared memory.

FIFO queues on buffers in memory, often integrated into the file system (UNIX *pipes*).

On weakly coupled systems (usually networks), message passing seems the *natural* communication mechanism. In a distributed system, communication needs lead to the use of specific resource management schemes for which message passing is appropriate. A resource will be in charge of a resource management process, with which any process that wants to access the resource must communicate following a **client-server** scheme, which allows expressing access to services through a request-response protocol. The client-server model can be implemented through a specific message passing mechanism, such as the UNIX *socket* interface, which relies on TCP/IP or UDP/IP protocols to communicate processes in a networked system.

But for independent physical memory spaces interfaces have also been designed to handle a common address space on a distributed physical memory system: **Distributed Shared Memory** (DSM) systems. The idea is to offer applications designed according to a shared memory model a support for their execution in a distributed environment.

In order to reduce the semantic difference between programs that use local services (through system calls) and programs that use remote services, mechanisms have been developed that encapsulate the details of addressing and synchronization of the sending/receiving of the request/response. Thus, in procedural languages, **remote procedure calls**, **RPC**s, provide a function call syntax as an interface for accessing services. In object-oriented languages, each resource is an object identified univocally in the distributed system, which is accessed by invoking the access methods defined for that object class. Access to objects is done through a **remote method invocation** interface (for example, Java RMI).

Table 1.4 summarizes the mechanisms that implement the communication models on both architectures.

Figure 1.2 summarizes the structure of communication in a distributed system. The different programming schemes are supported by the message passing interface characteristic of networked systems. Below we will briefly review the basic concepts related to communication based on message passing.

Communication model	Degree of coupling	
	Shared physical memory space	Independent physical memory spaces
Shared memory	Shared variables	Distributed Shared Memory (DSM)
Message passing	UNIX pipes	Sockets
Semantics of high-level languages	Calls to functions or procedures, invocation of methods on objects	RPC, RMI

Table 1.4. Communication mechanisms.

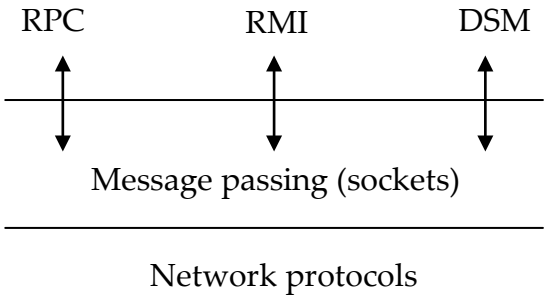


Figure 1.2: Structure of communication in a distributed system.

1.6.2 Communication based on message passing

Message passing comprises a set of mechanisms that allow processes to communicate through a communication channel or link, either directly, identifying the origin or destination process respectively in the *receive* or *send* primitives, or through a mailbox, which explicitly identifies the communication channel. In UNIX-like systems, the basic mechanism for message passing within a node is the *pipe*, with or without a name. To communicate processes between nodes, the problem of addressing arises. In this regard, UNIX *sockets* support two alternative forms of communication: identifying a socket within the file system (UNIX environment) or associating a communication **port** in a specific node (Internet environment). In the latter case, the explicit management of addressing is required.

Although the programming model of message passing for communication between nodes does not differ from that of message passing for communication within a node, the fact of having to rely on network protocols introduces

dependencies derived from the performance and reliability of the communication channel between nodes.

The main characteristics associated with a communication channel are the following:

- **Synchronization mode**. On the one hand, if the communication channel is busy, the send primitive can block the process until the channel is released (which also depends on whether it allows **buffering** or not) and can deposit the message (**blocking** or synchronous mode). The alternative (**non-blocking** mode) allows the process to continue even if the message could not be sent, transferring to the application the responsibility of managing the synchronization in the use of the user buffer where the sent message is located. These modes also apply to the receive primitive. UNIX sockets are in principle blocking, but they can be configured as non-blocking.

- **Reliability**. Regarding the reliability of the communication, the message passing mechanism depends on the support provided by the network. If implemented on a *secure* transport protocol, the communication is considered **reliable**, in the sense that the sender can trust that the receiver ends up receiving the message correctly or is informed otherwise. This is at the cost of a certain overload due to the need for the protocol to confirm the receptions. On the other hand, an **unreliable** mechanism will allow a less expensive communication, but delegates to the application the responsibility to verify the correctness of the communication. UNIX offers two forms of communication with sockets: *connection-oriented* communication, based on TCP/IP, reliable, and *datagram* communication, based on UDP/IP, unreliable.

- **Communication mode**. In distributed systems it is of great interest to support 1:N message passing primitives. **Broadcast** allows sending a message to all addresses accessible by the sender, and is used in local networks. A particular case of broadcast, the **multicast**, allows selecting a subset of addresses to which to send the message. Support for multicast is very useful in replicated systems, as will see later. As an example, the IP protocol reserves a set of addresses for multicast.

The development of the Internet is forcing the evolution of network level protocols. Thus, the IPv4 protocol, whose 32-bit addresses pose a problem of scalability, is giving way to IPv6, which specifies 128-bit addresses.

1.7 Open systems

A fundamental problem for the integration of resources in a distributed system is the heterogeneity of the systems that compose it, which affects the communication of data and the compatibility of applications and users. An essential basis for the construction of distributed systems over heterogeneous systems is the conception of the components as **open systems**. An open system is one that offers a **public specification** of its interface, which must also be assumed by the manufacturers. The diffusion of the system, normally subject to the laws of the market, determines to a large extent that the open system is effectively considered as such or not. In this sense, proposals for *official* open systems, defined by institutions and large consortiums, have failed, and, on the other hand, other systems of more modest conception have become **de facto standards**, such as the case of the TCP/IP protocol. On the contrary, for **closed systems**, the manufacturer does not provide the specification of their interfaces, so they are limited to proprietary configurations.

Open systems have the following properties [QUA93]:

- **Interoperability**. Ability to move information between machines through the network. Provided by standard communication protocols (e.g., TCP/IP) and specification languages (e.g., XML).
- **Application portability**. Ability to move application programs between machines of the system. Open operating systems provide a common interface based on official standards (e.g., POSIX), so applications are transportable at the source code level[18].
- **User portability**. Ability to allow a user to access the system from different machines without having to know the particular characteristics of each machine. Graphical user interfaces, by their very nature, provide this property to a large extent, although they may be different in different machines.

Thus, network protocols and the operating system fundamentally determine the open system characteristic. Since the specifications are public, on heterogeneous component systems, developers can implement one or more layers of software (often called *middleware*) that provides the single system

[18] Note that Java applications are transportable independent of the operating system, because a layer of middleware over the operating system (the JVM) provides the standard interface to the applications.

vision. As we will see, communication schemes and operating system structures have been designed to define open systems and efficiently support the distribution of resources.

As we have defined, open systems do not guarantee the portability of applications at the level of executable code. The source code must be compiled for each particular architecture. There is, however, the alternative of direct **interpretation** of the source code, at the cost of a notable loss of performance. In this sense, the Java language has become a de facto standard.

The problem of communicating software components written in different languages has also been addressed, by defining interfaces at each end and providing a common format. **Interface definition languages** (IDL) offer a notation for defining interfaces by specifying the input and output format of each argument. Examples of this type of languages are Sun XDR for remote procedure calls and CORBA IDL for remote method invocations. In recent years, languages such as HTML and XML have been imposed as de facto standards for the description of devices and services and the specification of interfaces.

The Internet is imposing new standards and communication styles. Thus, there is a general tendency to use HTTP as a protocol for accessing services, as an evolution of the classic RPC scheme. On the other hand, the distribution of services has caused an evolution of the classic client-server scheme towards *peer-to-peer* structures, where the client and server roles are interchangeable. In such systems, usually dynamic, the nodes indistinctly offer and request services, and sometimes cooperate in the search for services and in the routing of requests.

1.8 Summary

The objective of a distributed system is to integrate the resources and services connected by a communication network. From the point of view of the user and the applications, a distributed system provides a single machine vision and does not differ from a centralized one (Figure 1.3). On the other hand, from the point of view of the designer (the system as resource manager), the internal structure is conditioned by the physical distribution of resources (Figure 1.4).

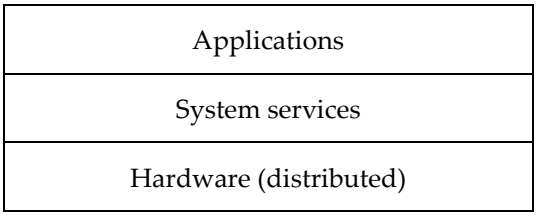


Figure 1.3: A distributed system from the point of view of the user.

Typically, the operating system integrates network services, which offer open communication protocols, such as TCP and UDP. On top of these, additional supports for distributed communication, such as RPC, RMI or DSM, and the specific services that provide the properties of the distributed system (*middleware* services), as is the case of time management, events and global state, on which the applications are based.

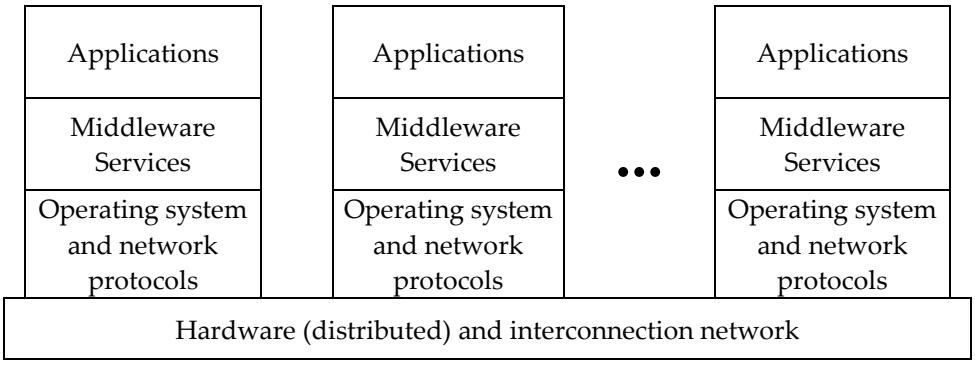


Figure 4: Structure of a distributed system.

1.9 Exercises

1 Does the *ftp* tool provide transparency in the identification and/or location of the files? Explain why. Give some example of tools and commands or system calls that do not provide these transparency properties. Give examples of tools that do provide them.

2 The IPv4 protocol is very limited in terms of the number of addresses it provides. Search and explain how IPv4 manages to extend its scalability based on conventions of use.

3 Explain through an example how by means of virtualization you can obtain transparency in the location of a service.

4 Enumerate the standard protocols that have had to be adopted so that a user who uses his PC in the campus cafeteria can access any web page. Use the Internet to search information.

Appendix A: **Remote procedure calls**

Message passing allows expressing the client-server model by specifying a request-response protocol about the service to which access is desired. Note that this protocol is based on a semantics oriented to the input-output. In contrast, access to resources in traditional operating systems, through the system call interface, is based on a function call-return semantics. Since transparency in the location of resources is a fundamental objective in distributed systems, it is evident that message passing does not provide the adequate semantic basis for accessing remote resources.

The **remote procedure call** mechanism (RPC), developed by Sun Microsystems, tries to eliminate that semantic leap. A client process specifies the access to a service through a function call syntax, for example:

result = rpc_service (parameters)

The name of the function can identify the service or be generic, in which case the RPCs interface will recognize the service by one of the parameters. Similarly, the server that runs the service uses the mechanism of RPCs to serve requests through a function that allows it to block waiting for requests. The complete procedure for executing an RPC is as follows (Figure 1.A1):

(1) Identified as remote the requested service, the RPC[19] mechanism invoked by the client process locates the server that supports it.

(2) Parameters are packaged in a standard format (for example, XDR[20]) to form the body of a message. This process is called **serialization** (*marshaling*).

(3) The operating system's network interface is used to send a message that contains the service request to the server (for example, via UDP/IP).

(4) The client process is blocked waiting for the response.

[19] It is called *stub*. It is the substitute function or representative in the client of the remote procedure.

[20] EXternal Data Representation. It has become a de facto standard.

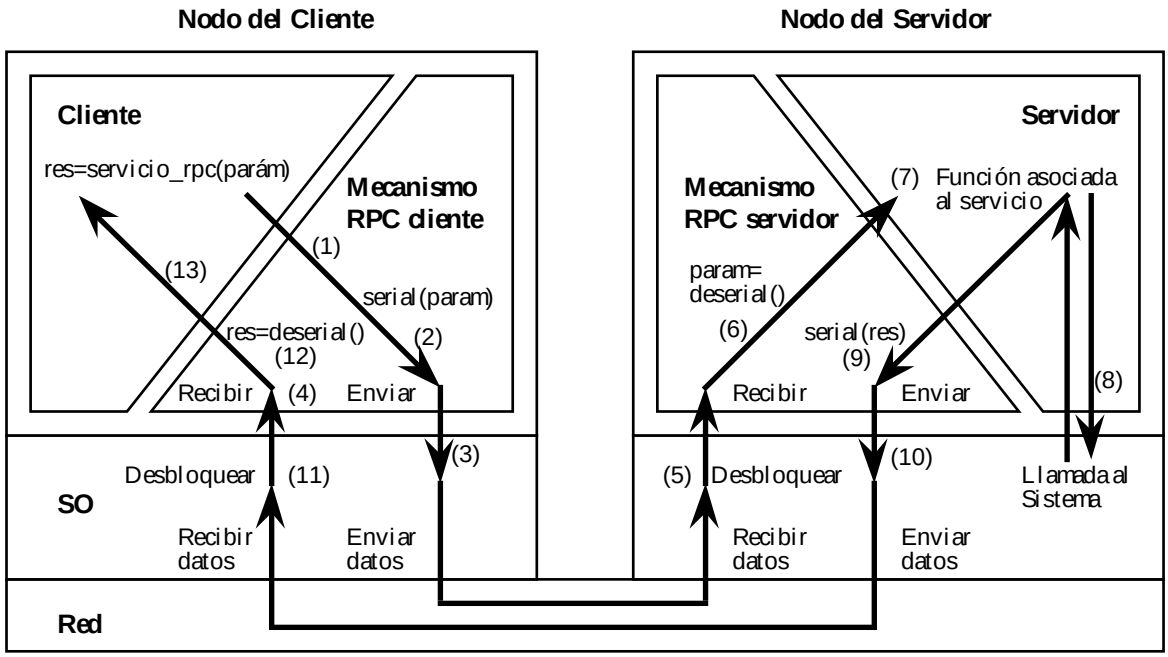


Figure 1.A1. Execution of an RPC.

(5) In the destination node, upon reception of the message, the operating system unblocks the server. If the server is multithread, a thread of execution will take care of the request.

(6) The RPC mechanism of the wait function of the server is executed, which unpacks the message (**deserialization**) to obtain the parameters of the service request and the identification of the origin.

(7) The RPC mechanism executes the function associated with the requested service, which will have been installed by the server in its initialization.

(8) The server function is executed through calls to the local system.

(9) When the function associated with the service terminates, the RPC mechanism serializes the result.

(10) The message with the result is sent to the client.

(11) The operating system of the client node unblocks the client upon reception of the result.

(12) The RPCs mechanism of the client executes the deserialization of the result.

(13) Returns the value to the program.

Note that the server will be blocked in the execution of a wait procedure. The RPC interface provides a specific function for this. In the initialization phase, the server will have installed the function(s) associated with the service(s).

The semantic level of a RPC library can be more or less high. Sun's RPC distinguishes three access levels. At the top level a series of *normal* functions and programs are supplied. At the intermediate level, a generic calling function is provided for the client, *callrpc()*, and two for the development of servers. At the lowest access level you can find specific functions that allow you to configure basic aspects of communication, such as asynchronous (non-blocking) calls, and permanent connection between client and server (*persistent binding*).

Appendix B: **Remote method invocation**

The invocation of remote methods (RMI) is to object-oriented programming what the RPCs are to procedural programming. Let's first review the concepts that are handled in the method invocation of the general object model, then focusing on the distributed object model.

The object model

In object-oriented programming languages, such as C++ and Java, an **object**, which encapsulates a set of data and **methods**, communicates with other objects by invoking their methods, which generally accept parameters and return results. Although these languages provide ways to directly reference the variables of the objects, in the distributed object model, which we will see later, the variables can only be accessed through methods.

Objects are accessed by reference[21]. In the invocation of a method of an object (which we will call *receiver* object), it is necessary to provide the reference to the receiver, the method and the parameters of the invocation. References to objects can be assigned to variables, passed as parameters or returned as results of an invocation.

An **interface** defines the format of access to a set of methods, that is to say, their names, types of the parameters, return values and exceptions (but not the implementation of the methods).

In the invocation of a method, the receiving object executes the method and returns the control to the invoking object, returning a result. As a result, the state of the receiver may change. Invocations to other objects can also be triggered. During the execution of the method, error conditions (**exceptions**) may occur.

The distributed object model

In a distributed system, the objects of the applications can be distributed among the nodes of the system. As in the case of RPCs, **remote method invocation** usually follows a client-server architecture, where the receiving objects are

[21] This *reference* concept should not be confused with the *pointer to memory* of C.

managed by the servers of these objects. The invocation maintains transparency in the location of the object.

In the distributed object model, the processes consist of objects that communicate through local or remote invocations. Local invocations are invocations to methods of the same process, while remote invocations are to object methods of other processes, whether they are in the same node or in other nodes of the system. Some objects can only receive local invocations, while others, **remote objects**, can receive both local and remote invocations. To invoke a method of a remote object, an object must have access to the **remote object reference** of the receiver, which is a unique identifier in the distributed system.

Each remote object has a **remote interface** that specifies which of its methods can be invoked remotely. These methods will be implemented by the class of the remote object. In *Java RMI*, remote interfaces are defined like any other Java interface, simply by extending the *Remote* interface. *CORBA* provides an interface definition language (*CORBA IDL*) that allows classes of remote objects and client programs to be programmed in different languages.

In a remote invocation, exceptions related to the remote nature of the invocation, such as time-outs, may occur in addition to those of the general model.

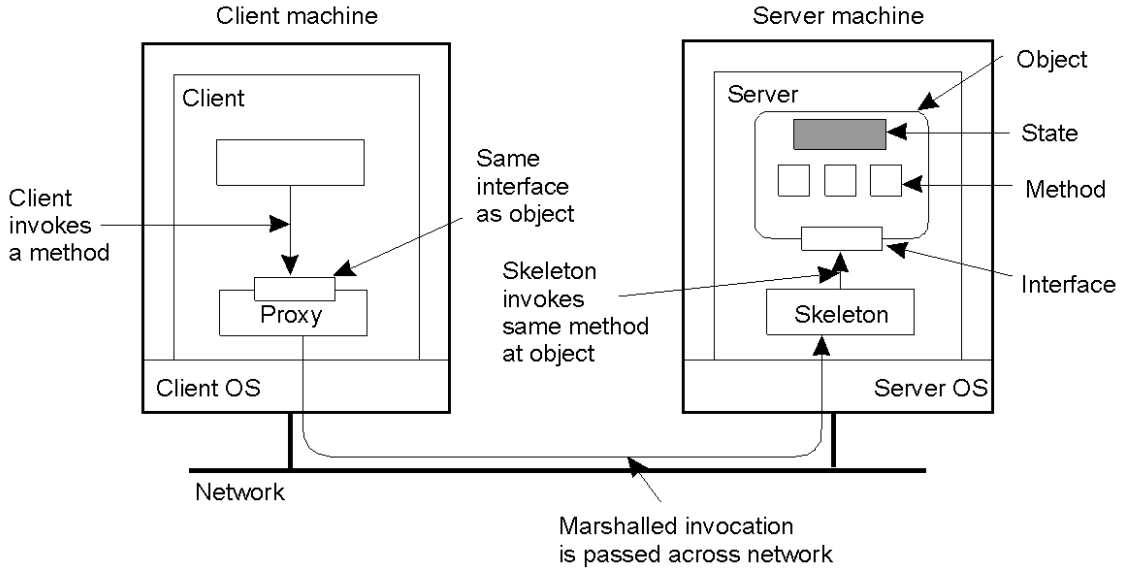


Figure 1.A2. Remote method invocation (taken from [TAN02]).

The implementation of the model of invocations of remote objects is analogous to that of RPCs (Figure 1.A2). On the client, a local representative of the class of the invoked object (*proxy*) transparently implements serialization and communication with the server that contains the receiver. On the server, the implementation of the receiver class (*skeleton*) manages the communication with the client and serialization, and invokes the corresponding method, treating any exceptions.

Java RMI

Java RMI extends the model of Java objects to support distributed objects in an integrated way in the language, making the invocation of remote methods transparent except for the fact that the client has to deal with remote exceptions and the server has to define as *Remote* the interface of the remote object.

Remote interfaces are defined by extending the *Remote* interface, provided by the *java.rmi* package. The parameters of a method are input, and the output is provided in the result of the invocation. Remote objects are passed by reference and local objects by value, through serialization. When the receiver does not have the implementation of the object that is passed by value, the Java virtual machine provides the automatic download of the corresponding class.

The nodes that host remote objects provide a name service that stores the references of remote objects, the *registry of remote objects* (*rmiregistry*).

In a distributed application in Java RMI, you must define the remote interfaces and implement the remote objects and the clients. Once the source files are compiled, the application is assembled in the following way:

1. *Proxies* (*stubs*, in Java-RMI terminology) of the remote classes are created by the RMI compiler (*rmic*).

2. Access to classes is specified for download and a security policy is established.

3. The registry of remote objects is started as a server process (*rmiregistry*).

4. The remote service is started and the clients are launched.